

Bedrock UX para Java

Documento de Engenharia e Design do Mod (GDD)

1. Visão Geral do Projeto

O objetivo deste projeto é desenvolver um Mod Client-Side (lado do cliente) para Minecraft Java Edition, construído sobre o ecossistema Fabric. O foco central é alcançar a paridade visual da interface gráfica (GUI) e da experiência de usuário (UX) em relação à edição Bedrock.

Como mods antigos que faziam essa função tornaram-se incompatíveis, este projeto será construído "do zero", utilizando códigos legados (como Bedrockify e Exordium) apenas como referência arquitetônica de injeção de Mixins, garantindo compatibilidade com as estruturas de renderização modernas da Mojang.

2. Escopo Arquitetônico e Ferramentas

Pilha de Tecnologia:

- **Linguagem:** Java 21 (Obrigatório para versões modernas do Minecraft).
- **Mod Loader:** Fabric (Leveza e facilidade de manipulação de injeções).
- **Mapeamentos:** Yarn (Padrão do Fabric) ou Mojmap.
- **IDE Recomendada:** IntelliJ IDEA com plugin "Minecraft Development".

3. Engenharia Reversa e Mixins (Como usar as bases antigas)

Para recriar as funções, precisaremos aplicar **Mixins**. Um Mixin é um fragmento de código que injetamos nas classes compiladas do jogo durante o carregamento. Analisando o código fonte de mods antigos, extrairemos a lógica matemática, mas aplicaremos nos novos métodos de renderização.

3.1. Implementação da "Paper Doll" (Boneco 3D)

O recurso visual mais importante. No Bedrock, o personagem é renderizado na interface. Para fazer isso no Java, precisamos interceptar a classe responsável pelo HUD do jogador e utilizar o sistema de renderização de entidades.

Classe Alvo: `net.minecraft.client.gui.hud.InGameHud`

```
@Mixin(InGameHud.class) public class PaperDollMixin { @Inject(method = "render", at = @At("TAIL")) private void renderPaperDoll(DrawContext context, float tickDelta, CallbackInfo ci) { // Lógica matemática para capturar estado de voo/corrida/nado // Renderizar a entidade do LocalPlayer no X/Y estático da tela // Usar InventoryScreen.drawEntity() como referência geométrica } }
```

3.2. Menus e Animações de Transição

No Java original, mudar de tela simplesmente descarta um objeto de tela e renderiza outro instantaneamente. No Bedrock, as telas deslizam (ease-in/ease-out).

A abordagem aqui é substituir o gerenciador de telas. Em vez de abrir a tela imediatamente, o mod salvará a imagem da "tela anterior" num buffer, sobrepondo-a com a "nova tela" através de uma função de interpolação linear (Lerp) atrelada ao frametime.

4. Especificações de Recursos (Features)

Módulo	Requisito Técnico	Desafio de Implementação
Coordenadas HUD	Desativar as coordenadas da tela F3. Renderizar uma caixa semi-transparente preta fixa no canto superior esquerdo com fonte sem sombra.	Garantir que as coordenadas não sobreponham buffs/debuffs do jogador.
Tela de Carregamento	Injetar na classe LevelLoadingScreen para exibir barras contínuas e ler um arquivo .json contendo as "Dicas de Jogo".	Sincronizar a animação da barra com as Chunks carregadas pelo Threading do Java.
Paridade de Água e Névoa	Alterar a matemática da classe BackgroundRenderer. O fog no Bedrock tem um início mais próximo do jogador, mas um fim mais denso.	Garantir compatibilidade com mods de otimização (Sodium/Iris), já que eles reescrevem o motor de renderização.

5. Roteiro de Desenvolvimento (Passo a Passo)

- Configuração do Ambiente:** Gerar um projeto no Fabric Template e importar para o IntelliJ. Compilar o mod vazio para garantir que o Gradle está configurado corretamente.
- Fase 1 (Elementos Estáticos):** Implementar primeiro a HUD de Coordenadas e alterar as cores e texturas básicas dos botões. (Baixa complexidade).

3. **Fase 2 (Injeção de Modelos):** Trabalhar na Paper Doll. Estudar como o jogo captura a matriz de rotação do personagem para reproduzir o movimento no canto da tela sem quebrar a proporção.
4. **Fase 3 (Lógica Dinâmica e Engine):** Implementar as transições de menu animadas e a substituição das telas de carregamento da Mojang.
5. **Fase 4 (Polimento):** Injeção dos pacotes de áudio (substituir sons de clique e botões), testes de compatibilidade com Sodium (importante para framerate) e publicação.

6. Considerações Finais

Ao se basear no código do BedrockWaters ou Exordium no GitHub, sempre verifique as assinaturas de métodos antigas e atualize-as utilizando a documentação do Fabric (FabricMC Docs). O segredo deste projeto está em manter os Mixins o menos invasivos possíveis, garantindo alta compatibilidade e estabilidade durante o gameplay.